MAHARASHTRA STATE BOARD OF TECHNICALEDUCATION

# GOVERNMENT POLYTECHNIC, JALGAON(0018)

**Program Name and Code : Computer Engineering (CO6K)**

**Course Name : Client Side Scripting (316005)**

**Course code : 316005**

**Academic Year : 2025-2026**

**Semester : Sixth Sem(3rd year)**

---

## A MICRO PROJECT ON

# Online Code Editor

| Sr. No. | Roll No. | Name of student | EnrollmentNo. | Seat No. |
|---|---|---|---|---|
| 1 | 26 | Tejal Sandip Patil | 23640220198 | 392105 |
| 2 | 77 | Sanghkirti Shaligram Sapkale | 23640220259 | 392156 |

**Project Guide By**

Prof. Varsha Mam

# MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION

# CERTIFICATE

This is to certify that **Ms. Tejal , Sanghkirti** Roll No. **26, 77** of **6th** Semester of Diploma in **Computer Engineering** of **Government Polytechnic, Jalgaon** (Code:0018) has completed the **Micro Project** satisfactorily in the subject **Client Side Scripting (316005)** for the Academic Year 2025- 2026 as prescribed in the curriculum.

| | |
|---|---|
| **Place : Jalgaon** | **Enrotllment No: 23640220198**<br>**23640220259** |
| **Date : …….** | **Exam Seat No: 392105**<br>**392156** |

| | | |
|---|---|---|
| **Prof. Varsha mam**<br>Subject Teacher | **Dr. P.P. Chaudhari**<br>Head of the Department | **Dr. P. M. Patil**<br>Principal |

Seal of Institution

# GOVTERNMENT POLYTECHNIC JALGAON

# -SUBMISSION-

We are **Tejal Patil, Sanghkirti Sapkale** Roll No. **26, 77** students of **6th** Semester of the **Computer Engineering** humbly submit that we have completed from time to time the Micro-Project work as described in this report by your own skills and study in academic year **2025-2026** as per instructions and guidance **Prof. Varsha Mam** that following students were associated with me for this work, however, quantum of my contribution has been approved by the Lecturer.

And that I have not copied the report on its any appreciable part from any other literature in contravention of the academic ethics.

**Date:** **Signature of Student: -**

| Roll No | Students Sign |
|---|---|
| 26 | |
| 77 | |

# Evaluation Sheet for the Micro Project

**Name of Students:** Tejal Patil, Sanghkirti Sapakle

**Enrollment No:** 23640220198, 23640220259

**Semester:** 6th Semester

**Course Title**: Client Side Scripting

**Course Code:** 316005

## Title of the Micro-Project
## Online Code Editor

## Micro-Project Evaluation Sheet

| RollNo. | Name of student | Marks out of 10 for performance in group activity | Marks outof 15 for Performance in oral/presentation | Total out of 25 |
|---|---|---|---|---|
| 26 | Tejal Sandip Patil | | | |
| 77 | Sanghkirti Shaligram Sapkale | | | |

# INDEX


| SR.NO. | CONTENT | PAGE NO. |
|---|---|---|
| 1. | Acknowledgement | 7 |
| 2. | Introduction | 8 |
| 3. | Objectives | 9 |
| 4. | Tools And Technologies | 10 |
| 5. | Flowchart | 11 |
| 6. | Features | 12 |
| 7. | Application | 13 |
| 8. | Code | 15 |
| 9. | Output | 23 |
| 10. | Conclusion | 26 |
| 11. | Reference | 27 |

# WEEKLY PROGRESS REPORT

| SR.NO. | WEEK | ACTIVITY PERFORMED | SIGN OF GUIDE | DATE |
|---|---|---|---|---|
| 1 | First Week | Discussion and finalization of topic | | |
| 2 | | Preparation and submission of Abstract | | |
| 3 | | Literature Review | | |
| 4 | | Collection of Data | | |
| 5 | Second Week | Collection of Data | | |
| 6 | | Discussion and outline of Content | | |
| 7 | | Formulation of Content | | |
| 8 | | Editing and proof Reading of Content | | |
| 9 | Third Week | Compilation of Report and Presentation | | |
| 10 | | Seminar | | |
| 11 | | Viva voce | | |
| 12 | | Final submission of MicroProject | | |

Prof. Varsha Mam
**(Sign of the faculty)**

# ACKNOWLEDGEMENT

This Micro project would not have been possible without considerable guidance and support. So, we would like to acknowledge those who have enable us to complete this project.

Firstly, we would like to thank our project guide, **Prof Varsha Mam** for providing the guideline with continuous advice and feedback throughout the duration of finishing this project. We also thank to the **Dr. P. M. Patil** (principal of Government Polytechnic Jalgaon) and **Dr. P.P Chaudhari** (HOD of computer department) for providing us the opportunity to embark on this project.

Secondly, we would also like to thank all other staff members of Computer department that we may called upon for assistance since the genesis of this project their opinion and suggestion have helped us in a realizing this project. Also not to be forgotten, we would like to Thank all our friends with the report valuable opinion and sharing ideas during the process of this project.

Finally, we would like to thank our families for their understanding, encouragement and support towards the completion of project.

Thank you so much.

# INTRODUCTION

An online code editor is a web-based platform that allows developers to write, edit, and test code directly from their browser without installing any software on their computer. These editors provide a convenient environment where users can work on programming projects from anywhere with an internet connection. Many online code editors support multiple programming languages and offer useful features such as syntax highlighting, auto-completion, and real-time error detection to make coding faster and more efficient.

One of the biggest advantages of online code editors is accessibility and collaboration. Developers can easily share their code with others through links, making it simple for teams to review, edit, and work on the same project together. This is especially useful for students, beginners, and remote teams who need a quick and flexible way to practice coding or collaborate on projects without complex setup processes.

Additionally, online code editors often include built-in tools such as compilers, debuggers, and version control integration. These features help programmers run and test their code instantly while keeping track of changes. As a result, online code editors have become an essential tool for modern development, learning programming, and building projects efficiently in a cloud-based environment.

# OBJECTIVE

**1. Easy Access to Coding**

The main objective of an online code editor is to allow users to write and edit code directly in a web browser without installing any software. This makes coding accessible from any device with an internet connection.

**2. Support For Multiple Programming Languages**

Online code editors are designed to support different programming languages such as HTML, CSS, JavaScript, Python, C++, and more, allowing users to work on various types of projects in one place.

**3. Impove Coding Efficiency**

They provide features like syntax highlighting, auto-completion, and error detection which help programmers write code faster and with fewer mistakes.

**4. Instant Code Execution**

Another objective is to allow users to run and test their code immediately. Built-in compilers or interpreters help users see the output quickly without additional setup.

**5. Collaboration and Code Sharing**

Online code editors make it easy to share code with others through links. This helps teams collaborate on projects and allows teachers or mentors to review students' code.

# Tools And Technologies Used

**1. HTML (Hyper Text Markup Language)**

HTML is used to design the structure of the online code editor interface. It creates elements such as text areas, buttons, menus, and other components displayed on the webpage.

**2. CSS (Cascading Style Sheets)**

CSS is used to style the editor interface and make it visually appealing. It controls layout, colors, fonts, spacing, and responsiveness of the web page.

**3. JavaScript**

JavaScript adds functionality and interactivity to the online code editor. It helps in features like running code, displaying output, syntax highlighting, and managing user actions.

**4. Code Editor Libraries**

These libraries provide advanced code editing features such as syntax highlighting, line numbers, auto-completion, and code formatting inside the browser.

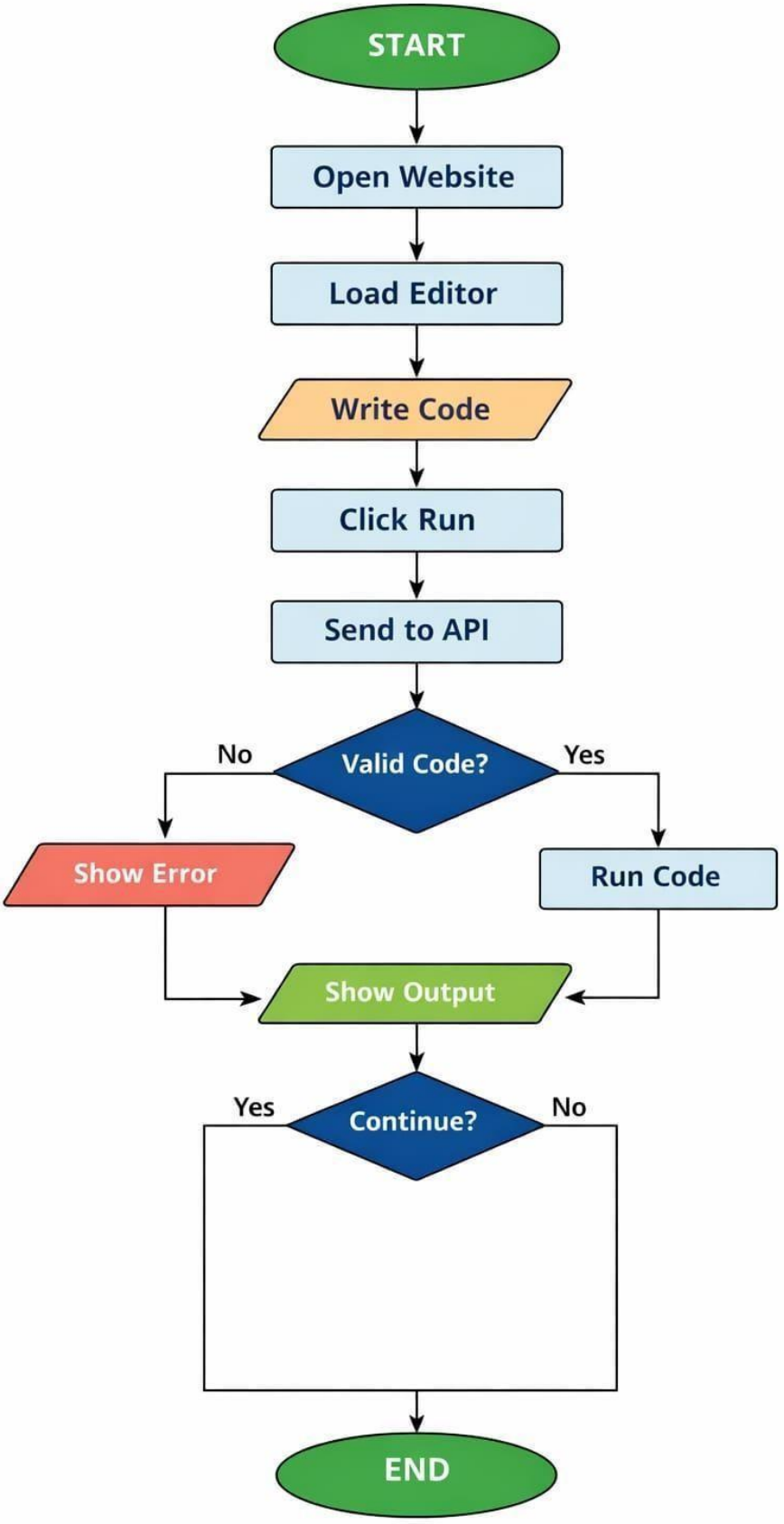
**5. Backend Technologies (Node.js / Python / PHP)**

Backend technologies are used to process code execution, manage server requests, store files, and handle user data.

**6. Web Browser**

The browser acts as the platform where the online code editor runs, allowing users to access and use it through the internet.

# Flowchart


START
Open Website
Load Editor
Write Code
Click Run
Send to API
Valid Code?
No
Yes
Show Error
Run Code
Show Output
Continue?
Yes
No
END

# Features

**1. Code Writing Interface**

- Provides a **text editor in the browser** where users can write and edit code.
- Supports syntax highlighting for different programming languages.

**2.. Multiple Language Support**

- Allows users to write and run programs in different languages such as **C, C++, Java, Python, and JavaScript**.

**3. Run / Compile Code**

- Users can **compile and execute the code directly online** without installing software on their computer.

**4. Syntax Highlighting**

- Different colors are used for **keywords, variables, and functions** to make the code easier to read.

**5. Error Detection**

- Displays **syntax errors and runtime errors** if the code has mistakes.

**6. Output Console**

- Shows the **result of the executed program** or the error messages.

**7. Auto Save**

- Automatically saves the user's code to prevent data loss.

**8. Code Sharing**

- Users can **share their code with others using a link**.

# Application

## 1. Learning Programming

Online code editors help **students and beginners learn programming easily**. They allow users to write, run, and test code directly in a web browser without installing any software. This makes it simple for beginners to practice languages like C, C++, Java, and Python. Many educational websites use online editors to teach programming concepts step by step.

## 2. Coding Practice Platforms

Many coding practice websites use online code editors so users can **solve programming problems and run their code instantly**. Students can practice algorithms, data structures, and logical problems. The editor shows the output and errors immediately, helping learners improve their coding skills.

## 3. Educational Institutions

Colleges and universities use online code editors in **programming laboratories and computer science courses**. Teachers can give assignments, and students can write and run programs online. This reduces the need to install compilers on every computer and makes lab management easier.

**4. Technical Interviews**

Many companies conduct **online coding tests and technical interviews** using online code editors. Candidates are asked to solve programming problems in a limited time. The editor compiles the code and shows the output so interviewers can evaluate the candidate's programming skills.

**5. Collaborative Coding**

Online code editors allow **multiple developers to work on the same code simultaneously**. Team members can edit code, review changes, and discuss solutions in real time. This feature is useful in team projects and software development environments.

**6. Quick Code Testing**

Developers often need to test small pieces of code quickly. Online code editors allow them to **write and run small programs instantly** without setting up a full development environment. This saves time and improves productivity.

**7. Web Development**

Online editors are commonly used for **developing and testing web applications**. Developers can write HTML, CSS, and JavaScript and immediately see the results in the browser. This makes it easier to design and test websites.

# Code

App.jsx

```
import { useState, useEffect } from "react";
import Editor from "./components/Editor";
import FileTabs from "./components/FileExplorer";
function App() {
const [loadingAI, setLoadingAI] = useState(false);
const [showHistory, setShowHistory] = useState(false);
const [chatHistory, setChatHistory] = useState([]);
const [html, setHtml] = useState("<h1>Hello Sanghkirti</h1>");
const [css, setCss] = useState("h1{color:purple}");
const [js, setJs] = useState("console.log('Working');");
const [darkMode, setDarkMode] = useState(true);
const [template, setTemplate] = useState("Custom");
const [lintErrors, setLintErrors] = useState({ html: [], css: [], js: [] });
useEffect(() => {
setDirty(true);
setShowReport(false);
if (live) updateLivePreview();
runLint();
}, [html, css, js, live]);
const updateLivePreview = () => {
const source = `
    <html>
      <head>
        <style>${css}</style>
      </head>
      <body>
        ${html}
        <script>
          try { ${js} } catch(error) {
            document.body.innerHTML += "<pre style='color:red'>" + error.message + "</pre>";
          }
        </script>
      </body>
```

```
      </html>
    setSrcDoc(source);
};
const runCode = () => updateLivePreview();
 const getCode = () => (active === "html" ? html : active === "css" ? css : js);
  const setCode = (value) => {
    if (active === "html") setHtml(value);
    if (active === "css") setCss(value);
    if (active === "js") setJs(value);
  };
  const runLint = () => {
    const jsIssues = [];
    const cssIssues = [];
    const htmlIssues = [];
// JS REAL ERROR
    try {
      new Function(js);
    } catch (err) {
      jsIssues.push({ line: "Unknown", msg: err.message });
    }
 const jsLines = js.split("\n");
    jsLines.forEach((line, idx) => {
    const t = line.trim();
if (line.includes("console.log"))
        jsIssues.push({ line: idx + 1, msg: "console.log detected" });
   if (line.includes("var"))
        jsIssues.push({ line: idx + 1, msg: "Use let/const instead of var" });
   if (line.includes("eval("))
        jsIssues.push({ line: idx + 1, msg: "Avoid eval()" });
  if (t && !t.endsWith(";") && !t.endsWith("{") && !t.endsWith("}") && !t.startsWith("//"))
        jsIssues.push({ line: idx + 1, msg: "Missing semicolon" });
   if (line.includes("=="))
        jsIssues.push({ line: idx + 1, msg: "Use === instead of ==" });
if (line.includes("undefined"))
        jsIssues.push({ line: idx + 1, msg: "Possible undefined usage" });
 if (line.includes("NaN"))
        jsIssues.push({ line: idx + 1, msg: "Check NaN handling" });
```

```
      if (line.includes("setTimeout(") && !line.includes(","))
        jsIssues.push({ line: idx + 1, msg: "setTimeout missing delay" });
    });
   // CSS
    if ((css.match(/{/g) || []).length !== (css.match(/}/g) || []).length) {
      cssIssues.push({ line: "Unknown", msg: "Unmatched { } brackets" });
    }
css.split("\n").forEach((line, idx) => {
      if (line.includes("!important"))
        cssIssues.push({ line: idx + 1, msg: "!important used" });
if (line.includes("px"))
        cssIssues.push({ line: idx + 1, msg: "px unit detected, use rem/em" });
 if (line.includes(";;"))
        cssIssues.push({ line: idx + 1, msg: "Extra semicolon" });
if (line.includes(": ;"))
        cssIssues.push({ line: idx + 1, msg: "Empty CSS value" });
    });
// HTML
    try {
      const parser = new DOMParser();
      const doc = parser.parseFromString(html, "text/html");
      if (doc.querySelector("parsererror")) {
        htmlIssues.push({ line: "Unknown", msg: "Invalid HTML structure" });
      }
    } catch {
      htmlIssues.push({ line: "Unknown", msg: "HTML parsing failed" });
    }
 html.split("\n").forEach((line, idx) => {
    if (line.includes("<script>"))
        htmlIssues.push({ line: idx + 1, msg: "Inline script detected" });
  if (line.includes("<font"))
        htmlIssues.push({ line: idx + 1, msg: "Deprecated <font>" });
 if (line.includes("<img") && !line.includes("alt="))
        htmlIssues.push({ line: idx + 1, msg: "Image missing alt" });
 if (line.includes("<a") && !line.includes("href"))
        htmlIssues.push({ line: idx + 1, msg: "Anchor missing href" });
```

```
      if (line.includes("<input") && !line.includes("type"))
        htmlIssues.push({ line: idx + 1, msg: "Input missing type" });
    });
    if ((html.match(/</g) || []).length !== (html.match(/>/g) || []).length) {
      htmlIssues.push({ line: "Unknown", msg: "Unbalanced HTML tags" });
    }
  setLintErrors({ js: jsIssues, css: cssIssues, html: htmlIssues });
  };
const scanCode = async () => {
  try {
  const res = await fetch("http://127.0.0.1:8000/api/scan/", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ html, css, js,lintErrors })// sending lintErrors from frontend
    });
    const data = await res.json();
setResult({
      score: data.score,
      issues: data.issues,
      fixes: data.fixes,
      types: data.issues.map(() => "AI")
    });
    setShowReport(true);
    setDirty(false);
    setChatHistory(prev=>[
      ...prev,
      {
        timestamp:new Date().toLocaleDateString(),
        result:data,
        html,css,js
      }
    ]);

    setNotify(true);
    setTimeout(() => setNotify(false),1000);
  } catch (err) {
    console.error(err);
  }
```

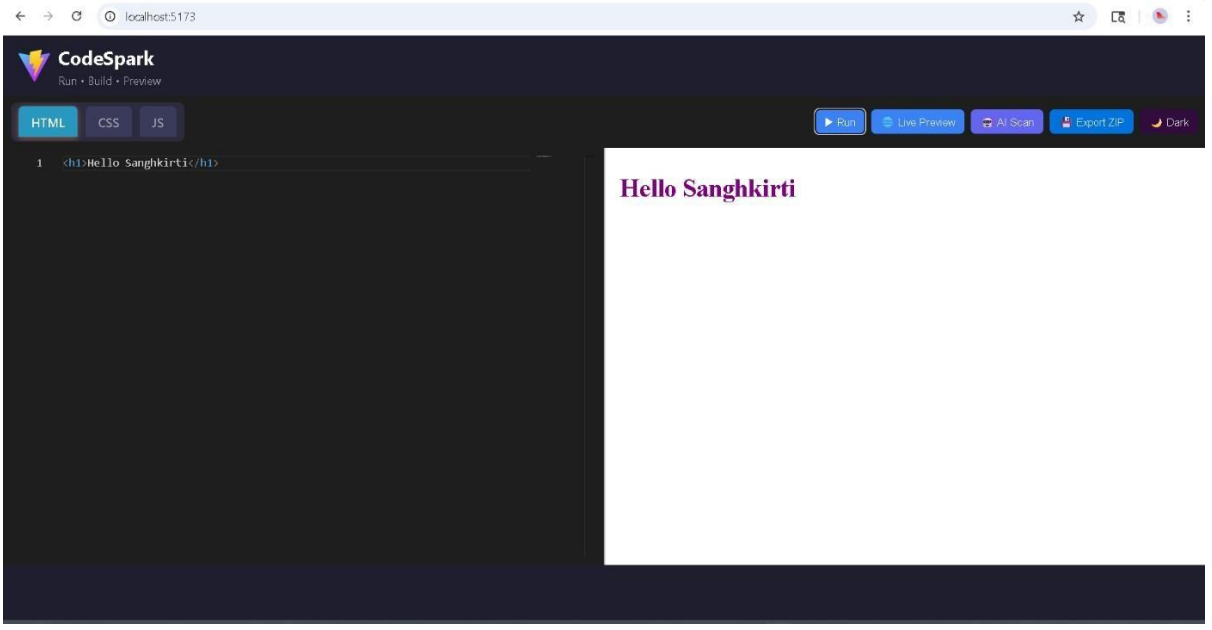
```
  setLoadingAI(false);
};
  const exportCode = async () => {
    const zip = new JSZip();
    zip.file("index.html", html);
    zip.file("style.css", css);
    zip.file("script.js", js);
    const content = await zip.generateAsync({ type: "blob" });
    saveAs(content, "code.zip");
  };
return (
    <div style={{ fontFamily: "Segoe UI, sans-serif", width: "100%", minHeight: "100vh", display:
"flex", flexDirection: "column", backgroundColor: darkMode ? "#1e1e2f" : "#f5f5f5", color:
darkMode ? "#fff" : "#000" }}>
      {/*Notification */}
      {notify && <div style={{ position:"fixed", top:"10px", left:"50%", transform:"translateX(-50%)",
zIndex:999, backgroundColor:"#2563eb", color:"#fff", padding:"10px 20px", borderRadius:"8px",
fontWeight:"600" }}>🤖 AI Report Generated! Check Below For Details</div>}
<Toolbar runCode={runCode} />
   {/* Tabs and Buttons */}
      <div style={{ display:"flex", justifyContent:"space-between", alignItems:"center", padding:"8px
10px", backgroundColor: darkMode ? "#1e1e1e" : "#ddd" }}>
        <FileTabs active={active} setActive={setActive} />
        <div style={{ display:"flex", gap:"8px" }}>
          <button onClick={runCode} style={{background:"#3b82f6", color:"#fff", padding:"6px 12px",
borderRadius:"6px", border:"none"}}>▶ Run</button>
          <button onClick={() => setLive(!live)} style={{background: live?"#10b981":"#3b82f6",
color:"#fff", padding:"6px 12px", borderRadius:"6px", border:"none"}}>📡 Live Preview</button>
          <button onClick={scanCode} disabled={!dirty} style={{background:dirty?"#6366f1":"#777",
color:"#fff", padding:"6px 12px", borderRadius:"6px", border:"none"}}>🤖 AI Scan</button>
          <button onClick={exportCode} style={{background:"#0073de", color:"#fff", padding:"6px
12px", borderRadius:"6px", border:"none"}}>📦 Export ZIP</button>
          <button onClick={() => setDarkMode(!darkMode)} style={{background:
darkMode?"#40005697":"#f3f4f6", color: darkMode?"#fff":"#000", padding:"6px 12px",
borderRadius:"6px", border:"none"}}>{darkMode?"🌙 Dark":"☀️ Light"}</button>
        </div> </div>
```

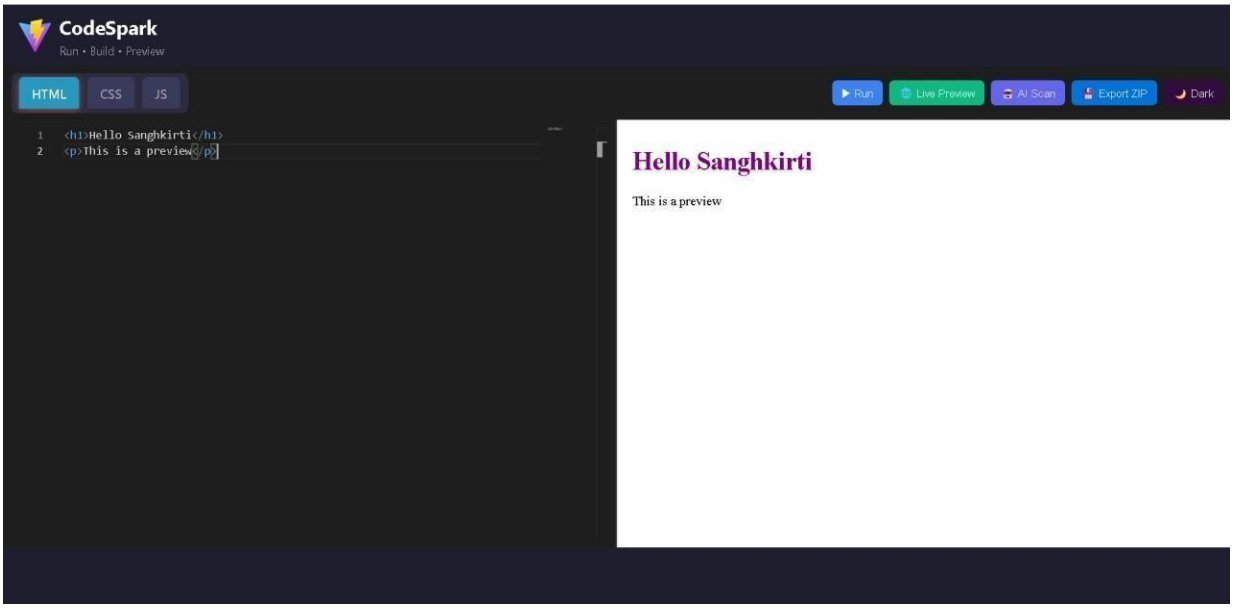
```
    {/* Editor + Preview */}
      <div style={{ display:"flex", width:"100%", minHeight:"500px" }}>
        <div style={{ width:"50%", backgroundColor: darkMode?"#1e1e1e":"#f5f5f5",
color:darkMode?"#fff":"#000", borderRight: darkMode?"2px solid #333":"2px solid #ccc",
padding:"10px", boxSizing:"border-box" }}>
          <Editor language={active} code={getCode()} setCode={setCode}
lintErrors={lintErrors[active]} />
        </div>
        <div style={{ width:"50%", backgroundColor:"#fff", borderLeft:"2px solid #ddd",
padding:"10px", boxSizing:"border-box" }}>
          <Preview srcDoc={srcDoc} />
        </div>
      </div>
 {/* AI Report */}
{showReport && result && (
 <div style={{
    width: "100%",
    padding: "20px",
    backgroundColor: darkMode ? "#0f172a" : "#f1f5f9",
    color: darkMode ? "#e2e8f0" : "#000",
    marginTop: "20px",
    borderRadius: "10px"
  }}>
<h2 style={{ textAlign: "center", marginBottom: "15px" }}>
      AI Code Review
    </h2>
<div style={{ textAlign: "left", marginBottom: "10px" }}>
  <button
    onClick={() => setShowHistory(!showHistory)}
    style={{
      background: "#6366f1",
      color: "#fff",
      padding:  "6px  12px",
      borderRadius:   "6px",
      border: "none",
      cursor: "pointer",
```

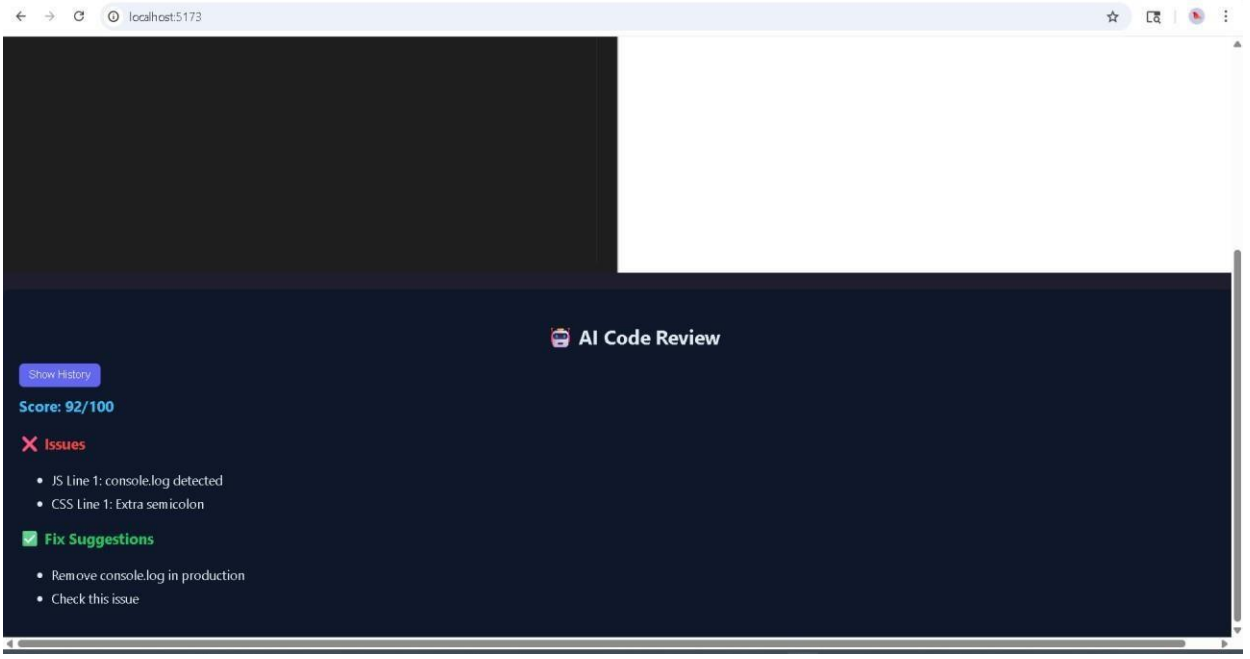
```
      textAlign:"left"
    }}
  {showHistory ? "Hide History" : "Show History"}
  </button>
</div>
{showHistory && chatHistory.length > 0 && (
  <div style={{
    width: "100%",
    padding: "15px",
    backgroundColor: darkMode ? "#111827" : "#f0f0f0",
    color: darkMode ? "#e5e7eb" : "#111",
    marginTop: "10px",
    borderRadius: "10px",
    maxHeight: "200px",
    overflowY: "auto",
    textAlign:"left"
  }}>
    <h3 style={{ marginBottom: "10px", textAlign:"left" }}>Scanned History</h3>
    <p style={{ paddingLeft: "20px" }}>
      {chatHistory.map((item, idx) => (
        <p key={idx} style={{ marginBottom: "10px" }}>
          <strong>{item.timestamp}</strong>: Score {item.result.score}/100
          <br />
          Issues: {item.result.issues.join(", ")}
          <br />
          Fixes: {item.result.fixes.join(", ")}
        </p>
      ))}
    </p></div>
    <div style={{ textAlign:
      "left", fontSize: "18px",
      marginBottom: "20px",
      fontWeight: "bold",
      color: "#38bdf8"
    }}>
```

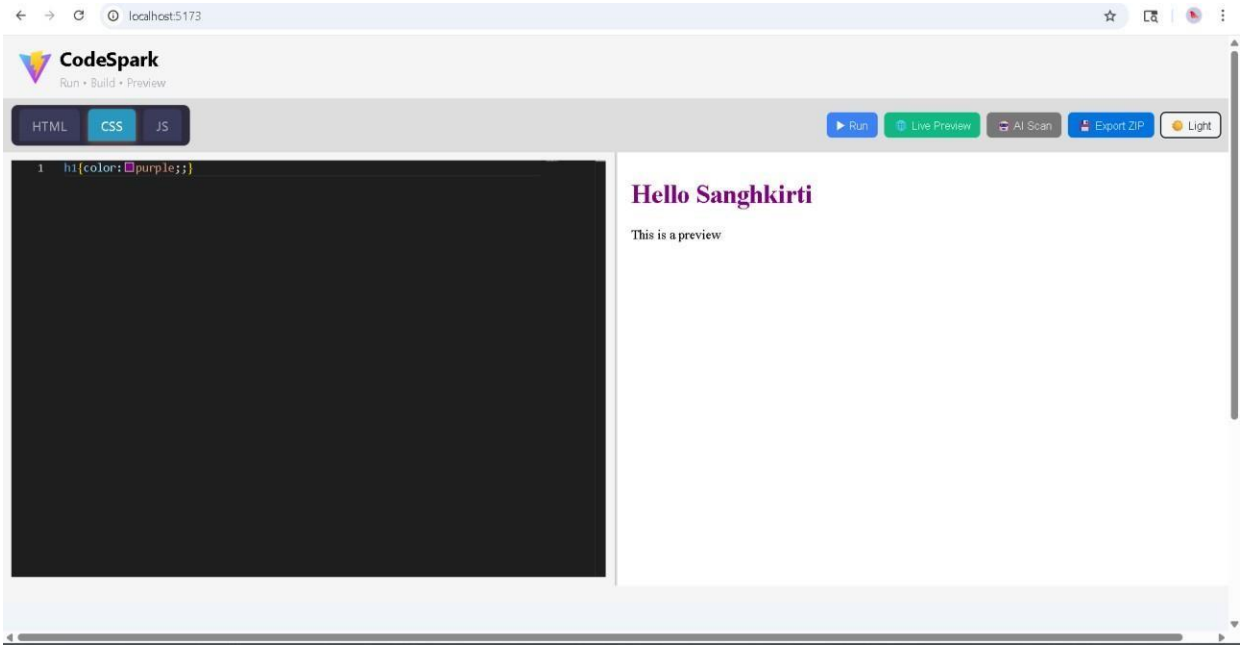
```
      Score: {result.score}/100
    </div>
    <div style={{ marginBottom: "15px" }}>
      <h3 style={{ color: "#ef4444" }}>+ Issues</h3>
      {result.issues.length > 0 ? (
        <ul>
          {result.issues.map((item, i) => (
            <li key={i} style={{ marginBottom: "6px" }}>
              {item}
            </li>
          ))}
        </ul>
      ) : (
        <p>No issues found $  </p>
      )}
    </div>
    <div>
      <h3 style={{ color: "#22c55e" }}>  Fix Suggestions</h3>
      {result.fixes.length > 0 ? (
        <ul>
          {result.fixes.map((item, i) => (
            <li key={i} style={{ marginBottom: "6px" }}>
              {item}
            </li>
          ))}
        </ul>
      ) : (
        <p>No fixes needed E  </p>
      )}
    </div>
  </div>
)}
</div>);
}
export default App;
```

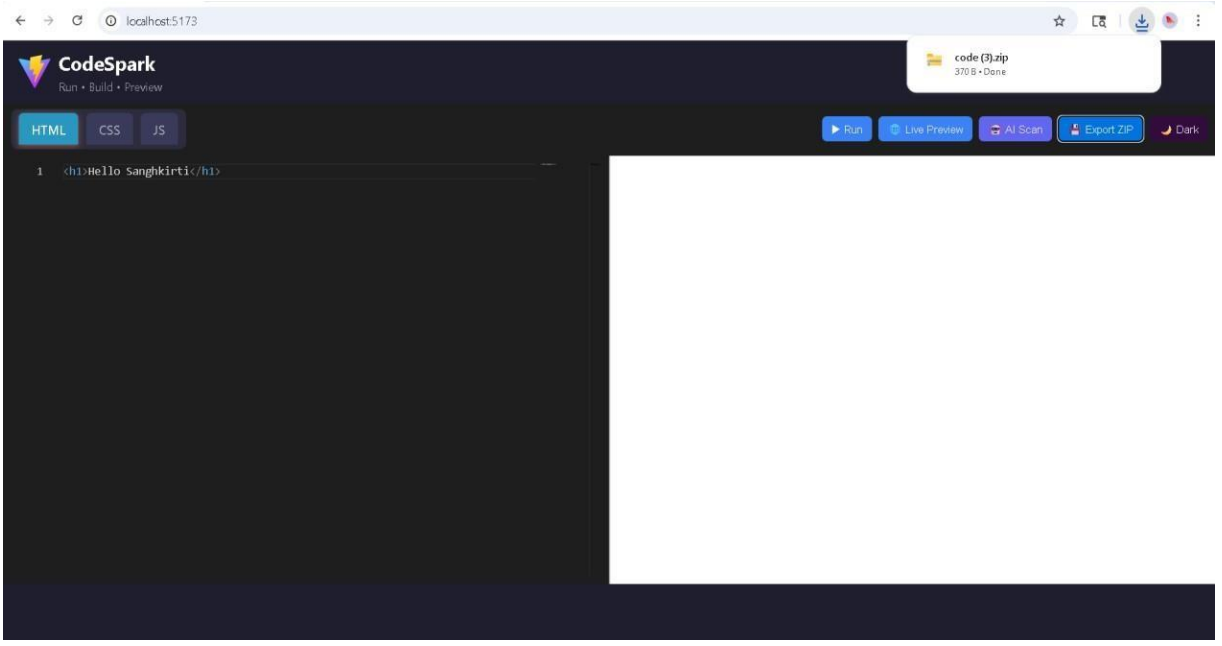
# OUTPUT


localhost:5173
CodeSpark
Run • Build • Preview
HTML
CSS
JS
Run
Live Preview
AI Scan
Export ZIP
Dark
1 <h1>Hello Sanghkirti</h1>
Hello Sanghkirti


CodeSpark
Run • Build • Preview
HTML
CSS
JS
Run
Live Preview
AI Scan
Export ZIP
Dark
1 <h1>Hello Sanghkirti</h1>
2 This is a preview
Hello Sanghkirti
This is a preview


localhost:5173
AI Code Review
Show History
Score: 92/100
Issues
JS Line 1: console.log detected
CSS Line 1: Extra semicolon
Fix Suggestions
Remove console.log in production
Check this issue


localhost:5173
CodeSpark
Run • Build • Preview
HTML
CSS
JS
Run
Live Preview
AI Scan
Export ZIP
Light
1 h1{color:purple;;}
Hello Sanghkirti
This is a preview


localhost:5173
CodeSpark
Run • Build • Preview
code (3).zip
370 B • Done
HTML
CSS
JS
Run
Live Preview
AI Scan
Export ZIP
Dark
1 <h1>Hello Sanghkirti</h1>

# CONCLUSION

An online code editor is a powerful tool that makes programming more accessible, flexible, and collaborative. It allows developers to write, run, and test code directly in a web browser without needing to install complex software on their devices. This convenience helps beginners start coding quickly and enables experienced developers to work from anywhere with an internet connection.

Another major advantage of online code editors is their support for collaboration and real-time sharing. Multiple users can work on the same project simultaneously, making them especially useful for team projects, learning environments, and coding interviews. Features like automatic saving, syntax highlighting, and integrated debugging tools also improve productivity and help reduce coding errors.

In conclusion, online code editors play an important role in modern software development by providing a simple, efficient, and collaborative coding environment. As technology continues to evolve, these tools will likely become even more advanced, helping developers build, test, and share applications more easily than ever before.

# Reference link


1. https://codepen.io
2. https://jsfiddle.net
3. https://codesandbox.io
4. https://www.codeinfoweb.com/tools/live-code-editor/
5. https://iarjset.com/papers/online-code-editor/